

Practical 4: Adapter Pattern

Part B – Analysis Questions

Q1. Why can't the notification system directly use the legacy email service?

Because the notification system expects a `send()` method while the legacy email service provides a `sendEmail()` method. Their interfaces are incompatible.

Q2. What problems arise when two classes have different interfaces?

- Integration becomes difficult.
- Code modifications are required.
- Tight coupling increases.
- Reusability decreases.
- Maintenance becomes harder.

Q3. What happens if multiple old systems need integration?

The application becomes difficult to manage because each legacy system may have a different interface. Adapters can be created for each system to provide a common interface.

Q4. Is modifying third-party code a good practice? Why?

No. Modifying third-party or legacy code can introduce bugs, increase maintenance costs, and create compatibility problems during future updates.

Q5. Which software quality attributes are affected?

- Maintainability
- Flexibility
- Reusability
- Extensibility
- Scalability

Part C: Problems in the Existing Design

Identify at least 3 problems:

1. The new notification system and old email service have incompatible interfaces.
2. The legacy email service cannot be used directly.
3. Modifying legacy or third-party code is risky and difficult.
4. Tight coupling makes maintenance difficult.
5. Adding more legacy services increases code complexity.

Practical 5: Facade

Part B – Analysis Questions

Q1. How many services are required to register a student?

Five services are required:

- Validation
- Roll Number Generation
- Save Student
- Department Assignment
- Notification

Q2. Why is the client tightly coupled with multiple services?

Because the client directly creates and calls every subsystem class, making the code dependent on many classes.

Q3. What happens if the registration process changes?

The client code must also be modified, increasing maintenance effort and the chance of errors.

Q4. How does Facade simplify the client code?

The Facade provides one simple method (`registerStudent()`) that internally coordinates all required services.

Q5. Which software quality attributes are affected?

- Maintainability
- Readability
- Reusability
- Flexibility
- Scalability

Part C – Problems in Existing Design

1. Tight coupling with multiple services.
2. Complex client code.
3. Difficult to maintain and modify.

Practical 6: Proxy

Part B – Analysis Questions

Q1. Who can currently access student records?

In the bad design, everyone can access the student records directly.

Q2. Why is this a security issue?

Confidential student information is exposed to unauthorized users, which can lead to privacy and security risks.

Q3. What happens if access checks are repeated everywhere?

The code becomes duplicated, difficult to maintain, and more prone to mistakes or inconsistent security checks.

Q4. Why should security be centralized?

Centralizing security ensures that access rules are managed in one place, making the system easier to maintain, update, and secure.

Q5. Which software quality attributes are affected?

- Security
- Maintainability
- Readability
- Reusability
- Scalability

Part C – Problems in Existing Design

1. No access control.
2. Confidential data is exposed.
3. Security logic is not centralized.

Practical 7: Decorator Pattern**Part B – Analysis Questions****Q1. Why are many report classes being created?**

Because every new feature combination requires creating another class.

Q2. What happens when a new feature is added?

More classes must be created, increasing complexity and maintenance effort.

Q3. Why does inheritance become difficult to maintain?

Inheritance creates many subclasses, making the code difficult to manage and extend.

Q4. How can Decorator add features dynamically?

Decorator wraps an existing object and adds new functionality without changing the original class.

Q5. Which software quality attributes are affected?

- Maintainability
- Flexibility
- Reusability
- Extensibility
- Readability

Part C – Problems in Existing Design

1. Too many subclasses.
2. Difficult maintenance.
3. Poor scalability.
4. Duplicate functionality.
5. Violates Open/Closed Principle.